SE(ECE)	Data Structures and Algorithms	
Experiment No: 10	Implement Graph using adjacency matrix. Perform DFS traversals.	Page: 1/3

Aim

To implement a Graph using an adjacency matrix representation and perform Depth First Search (DFS) traversal.

Objectives

1. To understand graph representation using adjacency matrix.
2. To learn and implement graph traversal techniques (DFS).
3. To analyze the time and space complexity of traversal algorithms.
4. To apply traversal algorithms in solving real-world problems like shortest path, connectivity, and network exploration**.**

Theory

Graphs

A graph data structure is a non-linear data structure used to represent relationships between pairs of objects.

A graph **G** is defined as:

G=(V,E)G = (V, E)G=(V,E)

where **V** is the set of vertices, and **E** is the set of edges.

Graphs can be:

- Directed / Undirected
- Weighted / Un-weighted

Adjacency Matrix Representation

- A 2D array of size V x V, where V is the number of vertices.
- If there is an edge from vertex i to vertex j, then matrix[i][j] = 1 (or weight if weighted graph), otherwise 0.

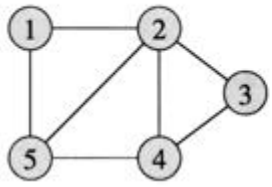
Adjacency List Representation

- Each vertex has a list of its adjacent vertices.
- Space efficient compared to adjacency matrix, especially for sparse graphs**.**

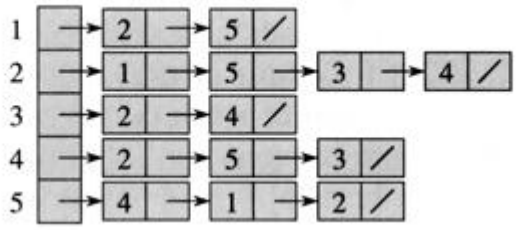
Traversals in Graphs

- Traversal means visiting each vertex of the graph systematically.
- The traversal method used in this experiment is:
    1. **DFS (Depth First Search):** explores as far as possible in one direction before backtracking.

SE(ECE)	Data Structures and Algorithms	
Experiment No: 10	Implement Graph using adjacency matrix. Perform DFS traversals.	Page: 2/3



Graph



Adjacency List

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

Adjacency Matrix

DFS (Depth First Search)

- Traverses graph by going deep into each branch before backtracking, using recursion or stack**.**
- Steps:
    1. Start from a source vertex, mark as visited.
    2. Visit an unvisited adjacent vertex, recursively continue.
    3. Backtrack when no unvisited adjacent vertices remain.
- **Applications:** Cycle detection, pathfinding, topological sorting, solving mazes.

Algorithms

DFS Algorithm (Recursive)

1. Start from source vertex, mark as visited.
2. Recursively visit all unvisited adjacent vertices.

SE(ECE)	Data Structures and Algorithms	
Experiment No: 10	Implement Graph using adjacency matrix. Perform DFS traversals.	Page: 3/3

Applications of DFS

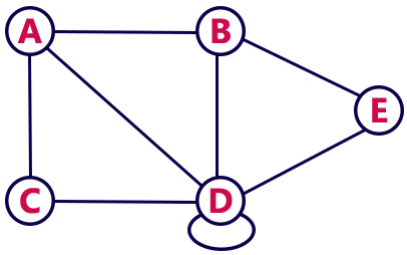
- **DFS:** Cycle detection, puzzle solving, maze exploration, scheduling.

References

1. Yedidyah Langsam, Moshe J Augenstein, Aaron M Tenenbaum – Data structures using C and C++ - PHI Publications (2nd Edition).
2. Ellis Horowitz, Sataraj Sahni- Fundamentals of Data Structures – Galgotia Books source.

Questions

Prepare adjacency matrix and adjacency list for the following graph.



Conclusion

A	C	O	T	Sign.